

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

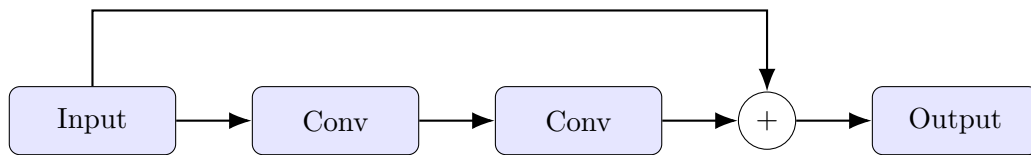


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

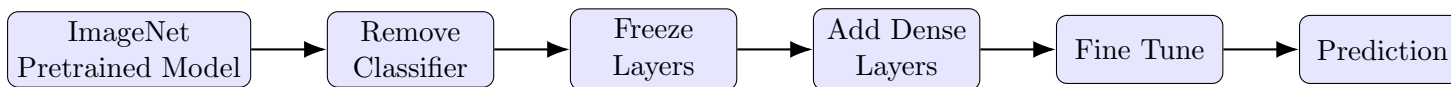


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Output

The following output was obtained after loading and preprocessing the CIFAR-10 dataset:

Training Images Shape : (50000, 32, 32, 3)

Training Labels Shape : (50000, 1)

Testing Images Shape : (10000, 32, 32, 3)

Testing Labels Shape : (10000, 1)

The dataset contains 50,000 training images and 10,000 testing images. Each image has dimensions 32×32 with 3 RGB channels. The pixel values were normalized to the range $[0, 1]$ by dividing the image values by 255.0.

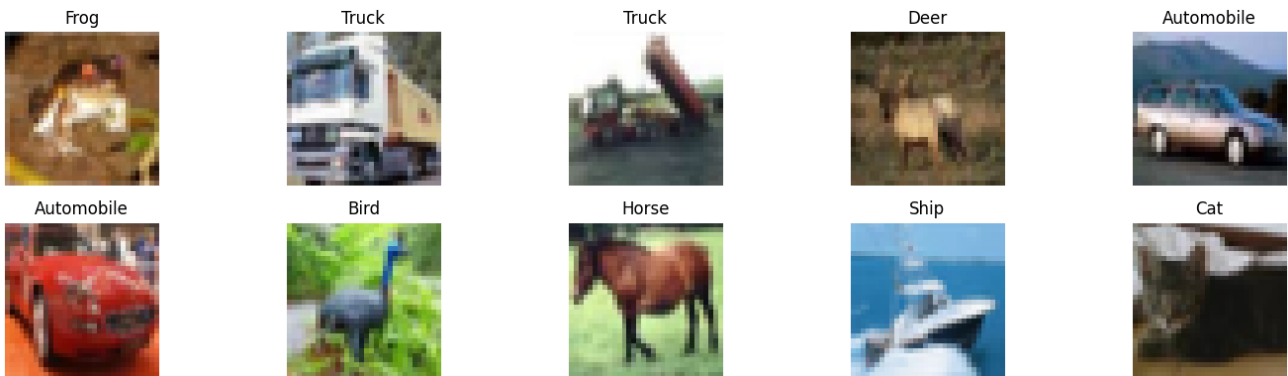


Figure 3: Ten sample images from the CIFAR-10 training dataset

Inference: The displayed samples represent different classes from the CIFAR-10 dataset. Normalizing the pixel values to the range $[0, 1]$ prepares the image data for subsequent neural network training.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Implementation

For this experiment, **MobileNetV2** was selected as the pretrained CNN architecture. The model was initialized with ImageNet pretrained weights and the original classification layer was removed using `include_top=False`. The input shape was set to (32, 32, 3) to match the CIFAR-10 images.

The convolutional base was frozen by setting `base_model.trainable = False`. A Global Average Pooling layer was then added, followed by a Dense layer containing 128 units with ReLU activation. Finally, a Dense output layer with 10 units and Softmax activation was added for the ten CIFAR-10 classes.

Model Architecture

The resulting transfer learning model consists of the following components:

- **Base Model:** MobileNetV2
- **Pretrained Weights:** ImageNet
- **Input Shape:** (32, 32, 3)
- **Original Classifier:** Removed
- **Convolutional Base:** Frozen
- **Global Average Pooling:** GlobalAveragePooling2D
- **Dense Layer:** 128 units, ReLU activation
- **Output Layer:** 10 units, Softmax activation

Model Summary Output

The model summary obtained from the notebook reported the following parameter counts:

Total params: 2,423,242 (9.24 MB)

Trainable params: 165,258 (645.54 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Table 1: MobileNetV2 Transfer Learning Model Parameters

Parameter Category	Value
Total Parameters	2,423,242 (9.24 MB)
Trainable Parameters	165,258 (645.54 KB)
Non-trainable Parameters	2,257,984 (8.61 MB)

Inference: MobileNetV2 was successfully adapted for the CIFAR-10 classification task using transfer learning. Since the convolutional base was frozen, only the newly added classification layers were trainable, resulting in 165,258 trainable parameters.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Model Compilation and Training

The MobileNetV2 transfer learning model was compiled using the Adam optimizer with a learning rate of 0.001. Categorical cross entropy was used as the loss function and accuracy was used as the evaluation metric. The model was trained for 10 epochs with a batch size of 32. The test dataset was supplied as the validation data during training.

Training Output

The exact training output obtained from the notebook is given below.

Epoch 1/10

1563/1563 - 77s - accuracy: 0.3004 - loss: 1.9321 - val_accuracy: 0.3263 - val_loss: 1.8667

Epoch 2/10

1563/1563 - 68s - accuracy: 0.3363 - loss: 1.8304 - val_accuracy: 0.3397 - val_loss: 1.8286

Epoch 3/10

1563/1563 - 68s - accuracy: 0.3488 - loss: 1.7910 - val_accuracy: 0.3472 - val_loss: 1.8113

Epoch 4/10

1563/1563 - 82s - accuracy: 0.3605 - loss: 1.7636 - val_accuracy: 0.3536 - val_loss: 1.8020

Epoch 5/10

1563/1563 - 79s - accuracy: 0.3675 - loss: 1.7403 - val_accuracy: 0.3477 - val_loss: 1.7971

Epoch 6/10

1563/1563 - 79s - accuracy: 0.3747 - loss: 1.7215 - val_accuracy: 0.3573 - val_loss: 1.7947

Epoch 7/10

1563/1563 - 69s - accuracy: 0.3802 - loss: 1.7057 - val_accuracy: 0.3587 - val_loss: 1.7908

Epoch 8/10

1563/1563 - 80s - accuracy: 0.3868 - loss: 1.6900 - val_accuracy: 0.3566 - val_loss: 1.7929

Epoch 9/10

1563/1563 - 67s - accuracy: 0.3904 - loss: 1.6781 - val_accuracy: 0.3581 - val_loss: 1.7908

Epoch 10/10

1563/1563 - 77s - accuracy: 0.3963 - loss: 1.6655 - val_accuracy: 0.3554 - val_loss: 1.7987

Training and Validation Performance

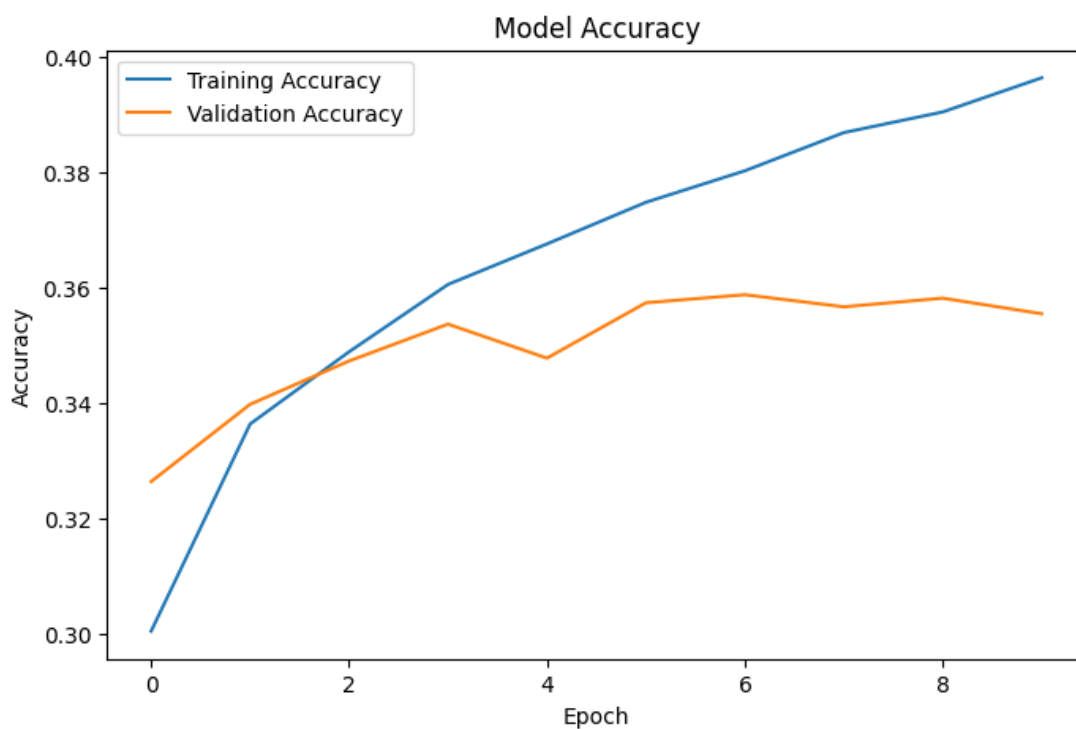


Figure 4: Training accuracy over the epochs

Inference: The training accuracy increased from 0.3004 in the first epoch to 0.3963 in the tenth epoch, showing gradual improvement during training.

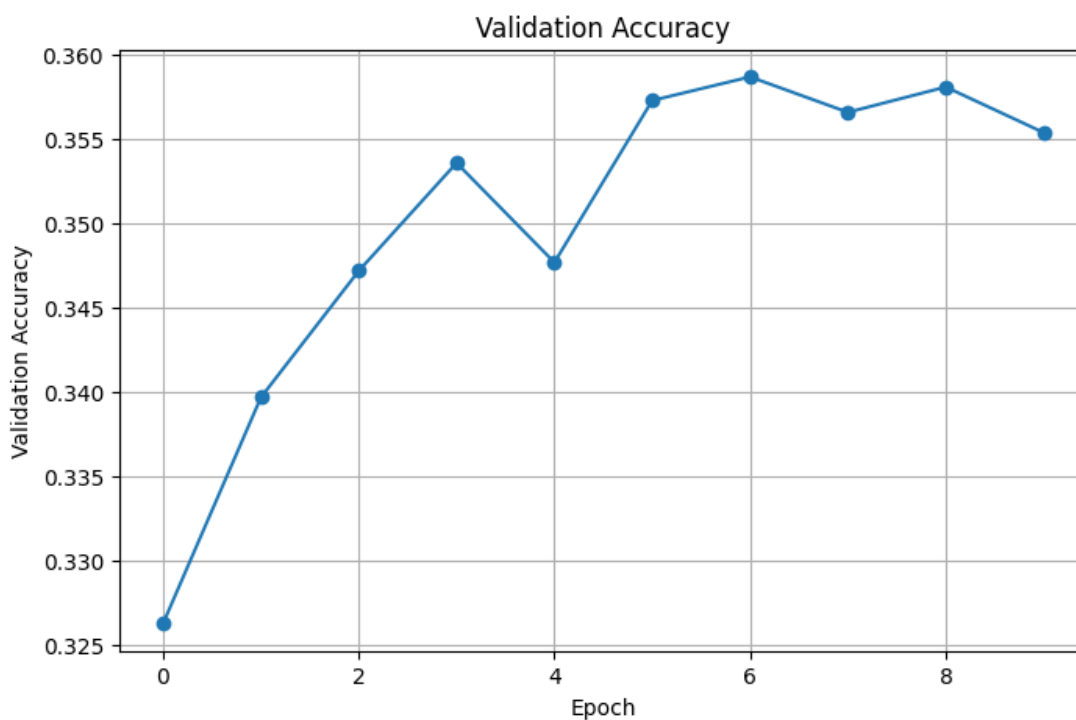


Figure 5: Validation accuracy over the epochs

Inference: The validation accuracy increased from 0.3263 in the first epoch and reached a maximum recorded value of 0.3587 at epoch 7. It was 0.3554 at the end of the tenth epoch.

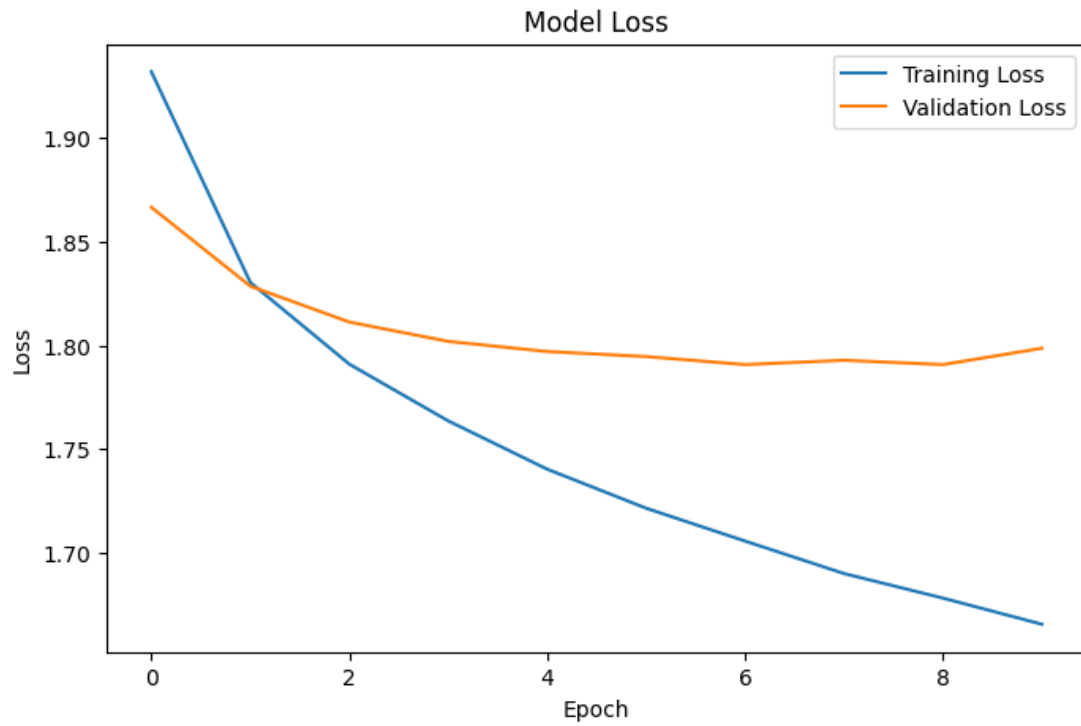


Figure 6: Training loss over the epochs

Inference: The training loss decreased from 1.9321 in the first epoch to 1.6655 in the tenth epoch, indicating that the model's training loss reduced progressively.

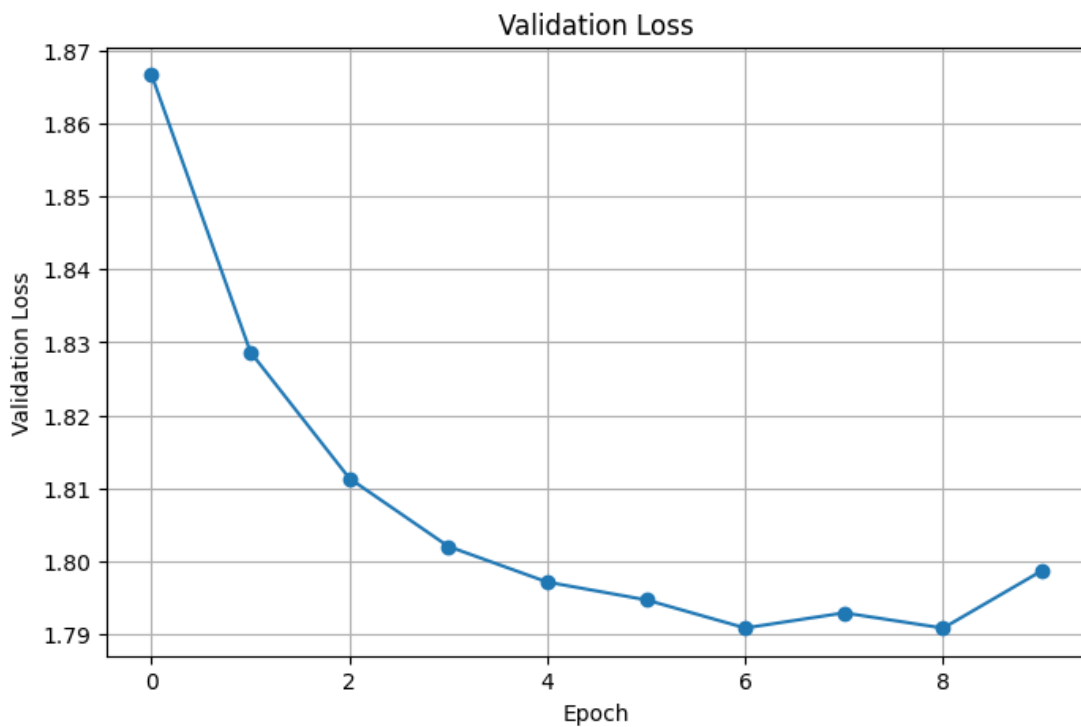


Figure 7: Validation loss over the epochs

Inference: The validation loss decreased from 1.8667 in the first epoch to 1.7908 at epoch 7, after which it showed small fluctuations and ended at 1.7987 in epoch 10.

Test Evaluation

After training, the model was evaluated on the test dataset. The exact output obtained from the notebook was:

```
313/313 - 11s - accuracy: 0.3554 - loss: 1.7987
```

```
Test Loss : 1.7987
```

```
Test Accuracy : 0.3554
```

Thus, the recorded test loss was **1.7987** and the recorded test accuracy was **0.3554**. :contentReference[oaicite:3]index=3

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Fine Tuning Procedure

For fine tuning, the pretrained MobileNetV2 base model was made trainable. All layers except the last 20 layers of the base model were kept frozen. Thus, only the last 20 layers of the pretrained base model were allowed to update their weights during fine tuning.

The model was recompiled using the Adam optimizer with a learning rate of 0.0001 and categorical cross entropy as the loss function. The model was then trained for 5 additional epochs with a batch size of 32.

Fine Tuning Training Output

Epoch 1/5

```
1563/1563 - 153s - accuracy: 0.3197 - loss: 6.1044 - val_accuracy: 0.2000 - val_loss: 5.3056
```

Epoch 2/5

```
1563/1563 - 142s - accuracy: 0.4036 - loss: 2.0243 - val_accuracy: 0.3694 - val_loss: 2.0656
```

Epoch 3/5

```
1563/1563 - 142s - accuracy: 0.4685 - loss: 1.5819 - val_accuracy: 0.4893 - val_loss: 1.4926
```

Epoch 4/5

```
1563/1563 - 143s - accuracy: 0.5148 - loss: 1.3800 - val_accuracy: 0.5405 - val_loss: 1.3273
```

Epoch 5/5

```
1563/1563 - 141s - accuracy: 0.5524 - loss: 1.2841 - val_accuracy: 0.5697 - val_loss: 1.2627
```

During fine tuning, the training accuracy increased from 0.3197 in the first epoch to 0.5524 in the fifth epoch, while the validation accuracy increased from 0.2000 to 0.5697.

Accuracy Comparison Before and After Fine Tuning

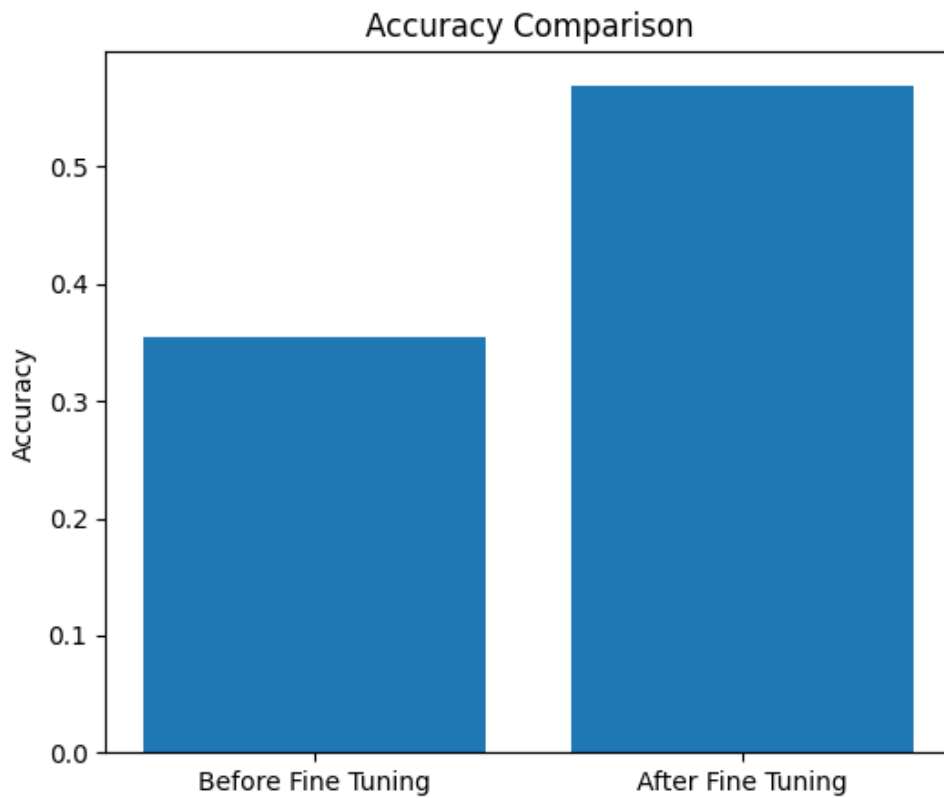


Figure 8: Classification accuracy before and after fine tuning

The exact evaluation output obtained after fine tuning was:

```
313/313 - 11s - accuracy: 0.5697 - loss: 1.2627
```

Accuracy Before Fine Tuning : 0.3554

Accuracy After Fine Tuning : 0.5697

The classification accuracy increased from **0.3554** before fine tuning to **0.5697** after fine tuning. Therefore, the fine tuned model achieved a higher classification accuracy on the test dataset.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Evaluation Metrics

The fine-tuned model was evaluated on the CIFAR-10 test dataset. Accuracy, weighted precision, weighted recall, and weighted F1-score were calculated using the predicted and true class labels.

The exact output obtained is:

Accuracy : 0.5697

Precision: 0.5686589099758084
Recall : 0.5697
F1 Score : 0.5665553918021374

Table 2: Evaluation Metrics of the Fine-Tuned Model

Metric	Value
Accuracy	0.5697
Precision	0.5686589099758084
Recall	0.5697
F1 Score	0.5665553918021374

The precision, recall, and F1-score were calculated using the **weighted** averaging method.

Confusion Matrix

The confusion matrix was generated to analyze the classification performance of the fine-tuned model across the ten CIFAR-10 classes.

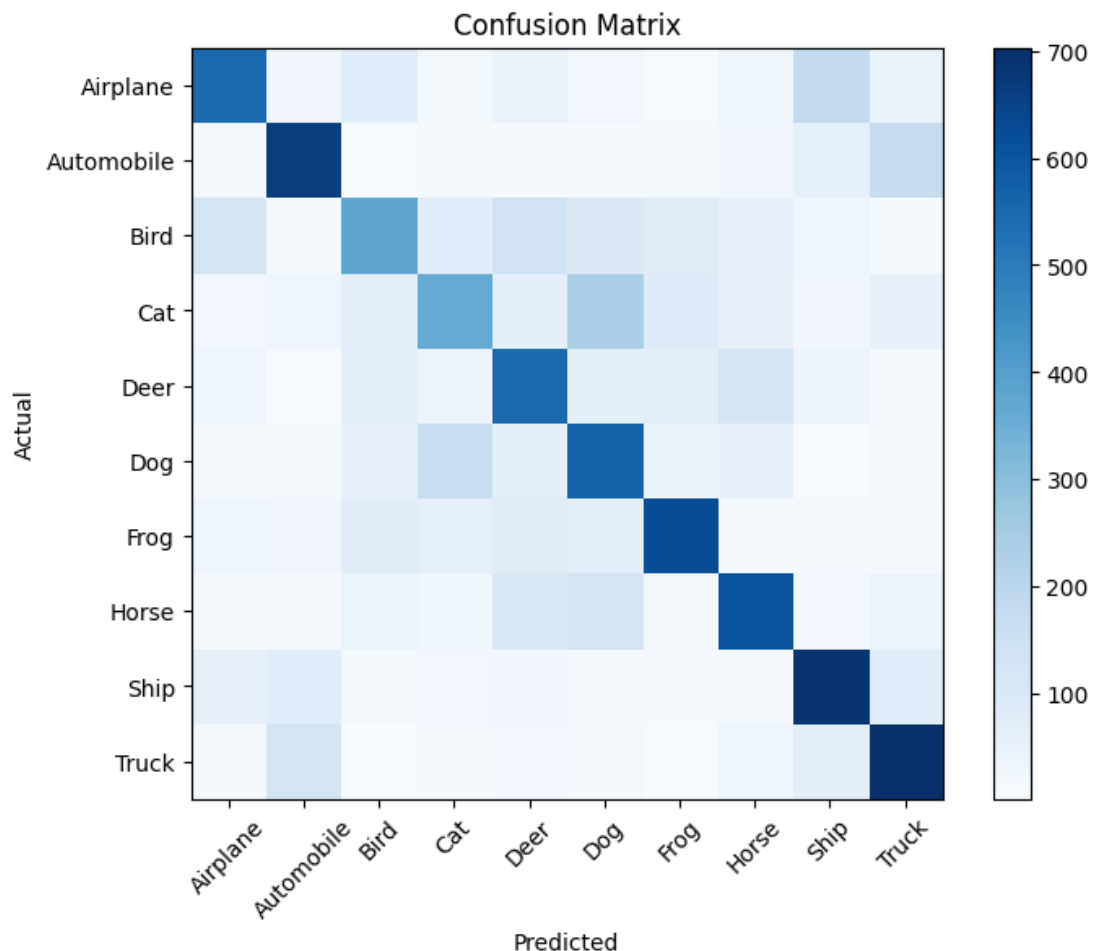


Figure 9: Confusion matrix of the fine-tuned MobileNetV2 model

Inference: The confusion matrix shows the distribution of correct and incorrect predictions across the ten CIFAR-10 classes. It provides class-wise information about which categories were correctly classified and which categories were confused with one another.

Classification Report

The classification report obtained is shown below.

	precision	recall	f1-score	support
Airplane	0.64	0.55	0.59	1000
Automobile	0.67	0.67	0.67	1000
Bird	0.48	0.38	0.42	1000
Cat	0.46	0.36	0.41	1000
Deer	0.50	0.55	0.52	1000
Dog	0.46	0.56	0.51	1000
Frog	0.65	0.62	0.63	1000
Horse	0.60	0.61	0.61	1000
Ship	0.61	0.69	0.65	1000
Truck	0.62	0.70	0.66	1000
accuracy			0.57	10000
macro avg	0.57	0.57	0.57	10000
weighted avg	0.57	0.57	0.57	10000

The classification report provides the precision, recall, F1-score, and support for each of the ten CIFAR-10 classes. **Inference:** The classification report shows that the model achieved different levels of performance across the ten classes. The highest class-wise recall was obtained for Truck with 0.70, while the lowest recall was obtained for Cat with 0.36.

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

Results

The classification accuracy obtained for the different hyperparameter settings is summarized below.

Table 3: Performance for Different Hyperparameter Settings

Hyperparameter Setting	Accuracy
Learning Rate = 0.001	0.3554
Learning Rate = 0.0001	0.6223000288009644
Batch Size = 16	0.6190999746322632
Batch Size = 32	0.3554
Batch Size = 64	0.5733000040054321
Epochs = 10	0.3554
Epochs = 20	0.6116999983787537
Optimizer = Adam	0.3554
Optimizer = SGD	0.6509000062942505
Dense Units = 128	0.3554
Dense Units = 256	0.3580999970436096
All Frozen Layers	0.3554
Partial Frozen Layers	0.6062999963760376

Observations

- The learning rate of 0.0001 produced an accuracy of 0.6223000288009644, which was higher than the accuracy obtained with a learning rate of 0.001.
- Increasing the batch size from 16 to 64 resulted in accuracies of 0.6190999746322632 and 0.5733000040054321 respectively, while the batch size of 32 produced an accuracy of 0.3554.
- Increasing the number of epochs from 10 to 20 resulted in an increase in accuracy from 0.3554 to 0.6116999983787537.
- SGD produced an accuracy of 0.6509000062942505, while the corresponding Adam setting produced an accuracy of 0.3554.
- Changing the number of Dense units from 128 to 256 resulted in accuracies of 0.3554 and 0.3580999970436096 respectively.
- Partial freezing resulted in an accuracy of 0.6062999963760376, compared with 0.3554 when all layers were frozen.

Inference

The results indicate that model performance was affected by the choice of training and architectural settings. Among the tested configurations, the SGD optimizer produced the highest recorded accuracy of 0.6509000062942505. Partial freezing also produced a higher accuracy than keeping all layers frozen. Increasing the training duration to 20 epochs and using a learning rate of 0.0001 also resulted in improved classification accuracy.

17. Mandatory Plots

1. Sample CIFAR-10 Images



Figure 10: Sample images from the CIFAR-10 dataset

Inference: The figure displays ten sample images from the CIFAR-10 dataset belonging to different object classes. The images show the variety of objects present in the dataset.

2. Training Accuracy

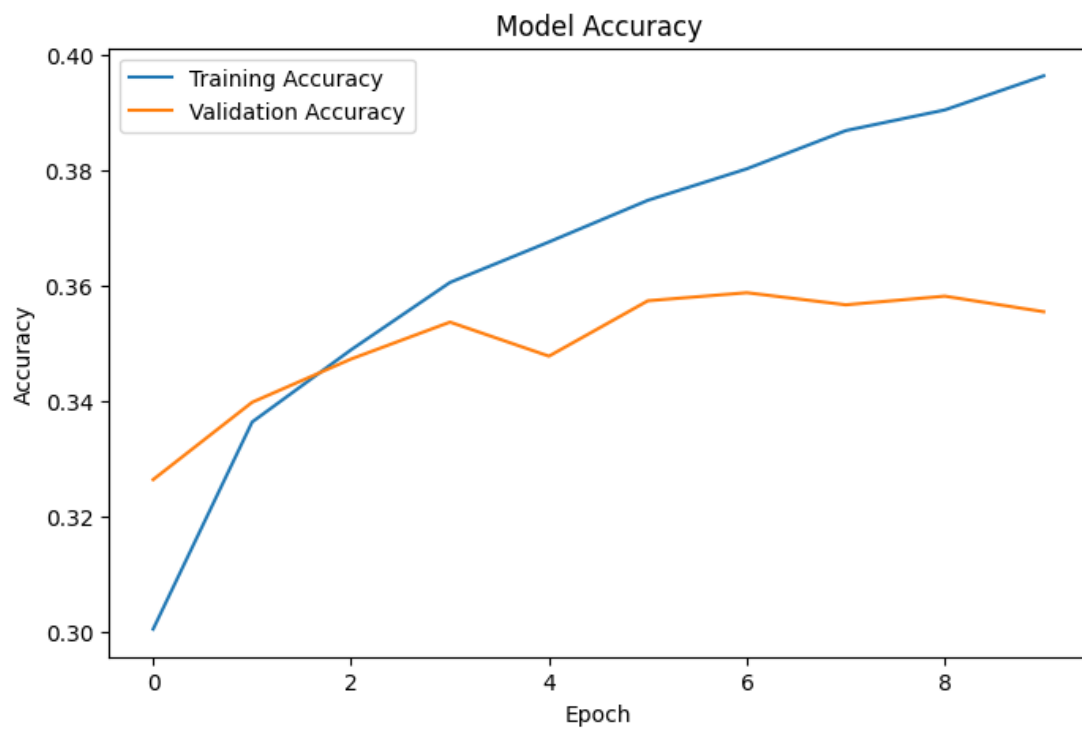


Figure 11: Training accuracy over epochs

Inference: The training accuracy gradually increases across the epochs. This indicates that the model learns the training data progressively during training.

3. Validation Accuracy

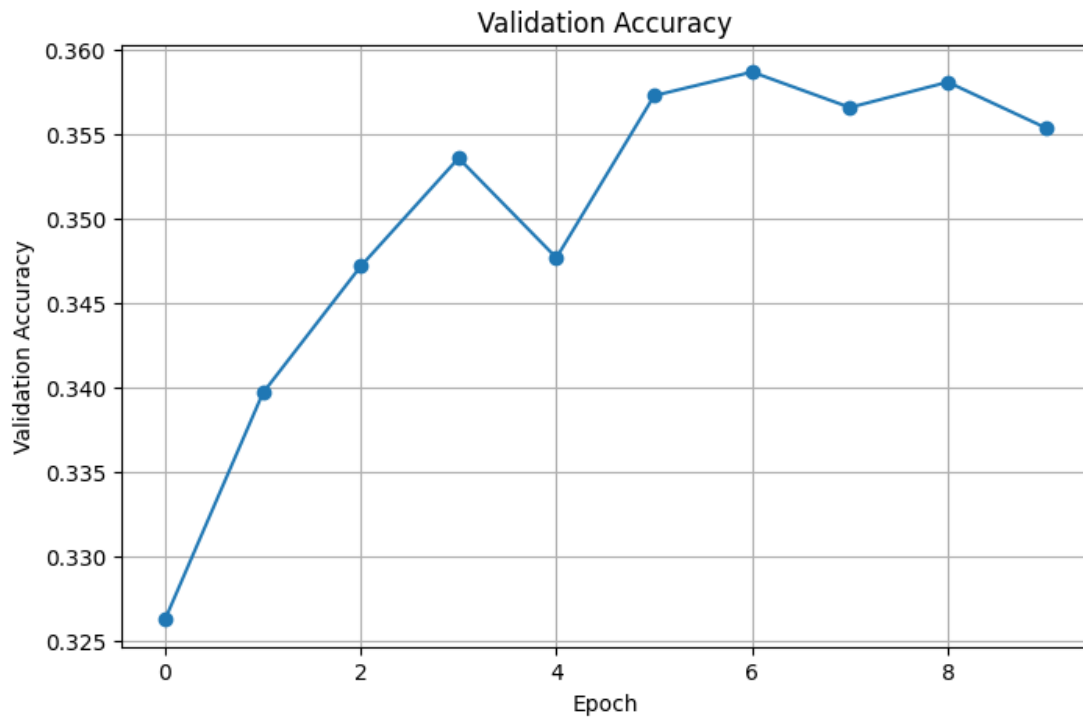


Figure 12: Validation accuracy over epochs

Inference: The validation accuracy improves during the initial epochs and then shows small fluctuations. This indicates that the model has learned useful features but its performance varies across epochs.

4. Training Loss

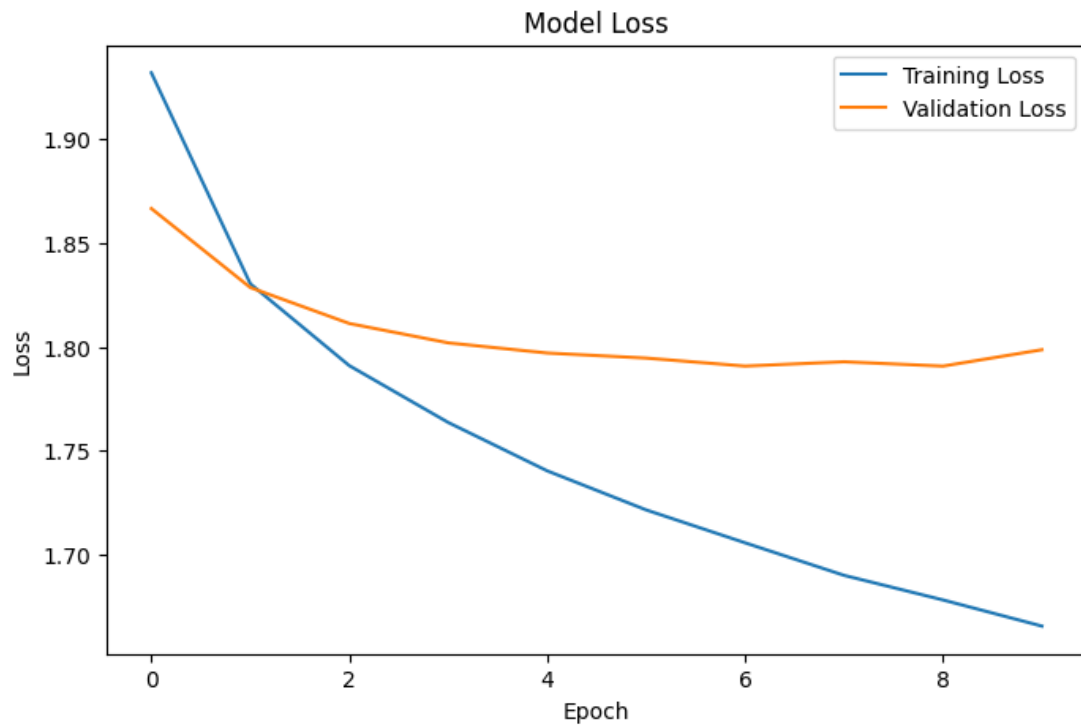


Figure 13: Training loss over epochs

Inference: The training loss decreases progressively as the number of epochs increases. This shows that the model is minimizing the training error during learning.

5. Validation Loss

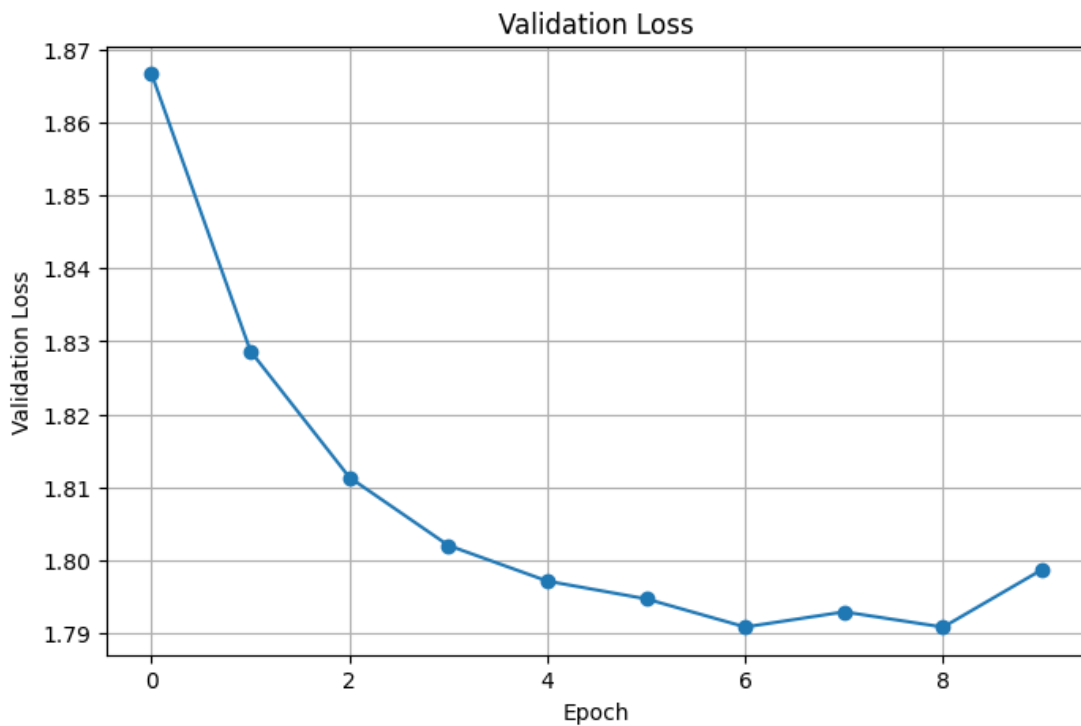


Figure 14: Validation loss over epochs

Inference: The validation loss decreases during the initial epochs and later shows small variations. This indicates that the model's generalization performance does not improve uniformly throughout training.

6. Confusion Matrix

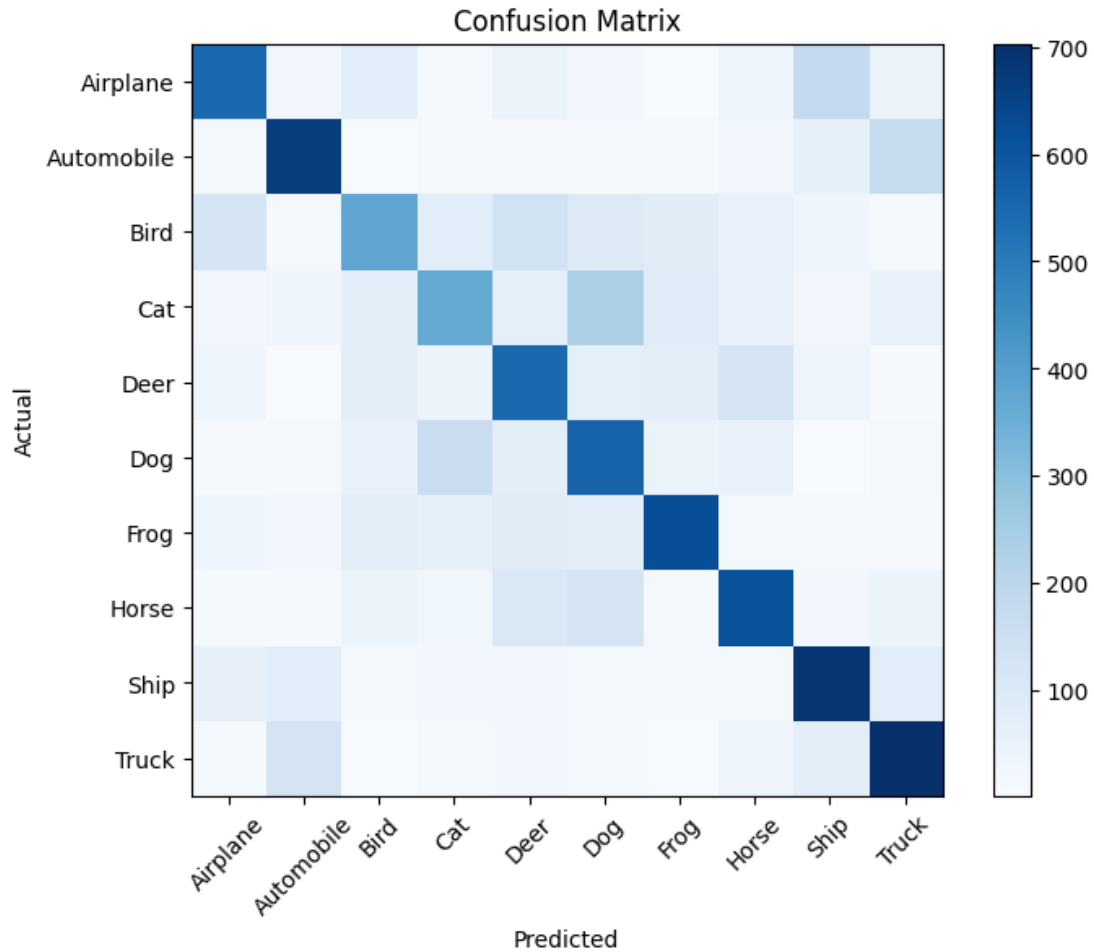


Figure 15: Confusion matrix for CIFAR-10 classification

Inference: The confusion matrix shows the classification performance for each of the ten CIFAR-10 classes. The diagonal elements represent correctly classified samples, while the off-diagonal elements represent misclassifications.

7. Misclassified Images

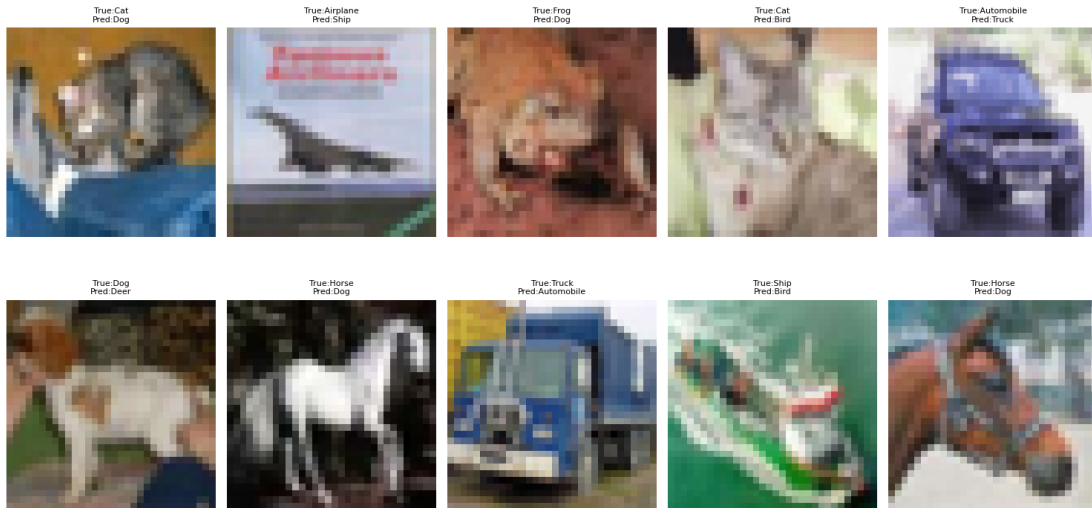


Figure 16: Examples of misclassified CIFAR-10 images

Inference: The figure presents examples where the predicted class differs from the actual class. These examples illustrate the types of images that remain challenging for the trained model.

18. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy	0.3963
Testing Accuracy	0.3554
Precision	0.5686589099758084
Recall	0.5697
F1-score	0.5665553918021374
Training Time	Not recorded
Total Parameters	2,423,242 (9.24 MB)

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time (seconds)
LeNet-5	83,126	52.99	37.83
AlexNet	36,926,730	59.95	191.15
VGG16	15,245,130	10.00	421.11
GoogleNet	930,394	78.19	262.00
ResNet50	23,608,202	43.45	299.13

Observation

The comparison shows that **GoogleNet achieved the highest test accuracy of 78.19%**, while LeNet-5 required the least training time. **AlexNet had the highest number of parameters**, whereas VGG16 showed the lowest accuracy among the compared architectures.

19. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**
AlexNet was a major breakthrough because it showed that deep CNNs could achieve very high accuracy for image classification. It introduced techniques such as ReLU activation, Dropout, GPU-based training and data augmentation, which helped in training deeper networks effectively.
2. **Why does VGG16 use only 3×3 convolution filters?**
VGG16 uses small 3×3 filters because multiple small filters can learn complex features while keeping the number of parameters lower than using larger filters. It also allows the network to become deeper and learn more detailed features.
3. **Explain the advantages of the Inception module.**
The Inception module performs convolutions using different filter sizes in parallel. This helps the network learn features at different scales while keeping the computational cost relatively low.
4. **What is the purpose of residual learning?**
Residual learning helps in training very deep neural networks. Instead of learning the complete mapping directly, the network learns the residual difference using skip connections, which also helps reduce the vanishing gradient problem.
5. **Differentiate LeNet and ResNet.**
LeNet is a relatively small and simple CNN mainly designed for handwritten digit recognition. ResNet is a much deeper network that uses residual or skip connections, allowing it to train effectively even with a large number of layers.
6. **What is Transfer Learning?**
Transfer learning means using a model that has already been trained on a large dataset and adapting it to a new problem. Instead of training the whole network from the beginning, the learned features can be reused and only the required layers are trained or fine-tuned.
7. **Why is fine tuning required?**
Fine tuning is used to adapt the pretrained features to the new dataset. After the initial training with frozen layers, some of the deeper convolution layers are unfrozen and trained with a smaller learning rate so that the model can learn features specific to the new task.
8. **Explain the difference between dilated convolution and transpose convolution.**
Dilated convolution increases the receptive field of a convolution without increasing the number of parameters significantly. Transpose convolution is mainly used to increase the spatial dimensions of feature maps and perform learnable upsampling.
9. **Why do pretrained models converge faster?**
Pretrained models already contain useful features learned from a large dataset. Therefore, they do not have to learn basic features such as edges and textures from the beginning, which usually allows them to converge faster.
10. **Compare the computational complexity of LeNet and ResNet.**
LeNet has a small number of layers and parameters, so it requires much less computation and memory. ResNet is much deeper and has many more parameters, making it computationally more expensive, but its residual connections allow it to train effectively and achieve better performance on complex image classification tasks.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

Source Code:

<https://colab.research.google.com/drive/14uhWrPla6D5461LUm3mHUPXwYYiuviId?usp=sharing>

<https://colab.research.google.com/drive/1gq6goHCVtfGX4SPwy5HDrjJiABziGw4C?usp=sharing>

https://github.com/LaasyaPriya-1238/DL_LAB

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>